

INTRODUCTION

The University Notice Management System is a mobile application developed to simplify the process of sharing and managing university notices digitally. In many colleges and universities, important announcements are still displayed on physical notice boards, making it difficult for students to access information quickly. This project provides a digital platform where notices can be created, managed, and viewed from anywhere using a mobile application.

The application has been developed using Flutter for the frontend and Laravel for the backend. MySQL is used as the database to store user information, notices, departments, and categories. The system supports different user roles such as Super Admin, Admin, and User, where each role has specific permissions and responsibilities.

The main purpose of this project is to reduce paperwork, improve communication between the administration and students, and provide a secure and efficient way to manage notices. Users can log in to the application, view notices based on their departments or categories, download PDF attachments, and access important information instantly. Administrators can create, update, and delete notices, while Super Admin has additional privileges to manage users and administrators.

This project also implements REST APIs to establish communication between the Flutter application and the Laravel backend. Images and PDF files are uploaded and managed through the backend, and all application data is stored securely in the MySQL database.

PROBLEM STATEMENT

In many educational institutions, important notices are still shared through physical notice boards or informal communication channels. This traditional approach creates several problems for both students and the administration. Students may miss important announcements if they are absent from campus, and administrators need to spend extra time printing, updating, and replacing notices.

Managing a large number of notices manually is also difficult. There is no proper way to organize notices based on departments or categories, and searching for old notices becomes almost impossible. Sharing notices with all students instantly is another major challenge.

The existing system also lacks role-based management. There is no proper control over who can create, edit, or delete notices, which may lead to unauthorized changes or confusion.

To overcome these problems, the University Notice Management System was developed. The application provides a digital platform where notices can be managed efficiently and accessed by students at any time. Administrators can create and update notices quickly, while students can view the latest announcements, download PDF attachments, and search notices based on departments or categories. This improves communication, reduces paperwork, saves time, and makes the overall notice management process more efficient.

OBJECTIVES

The main objective of this project is to develop a digital notice management system that allows universities and colleges to manage notices efficiently. The application is designed to provide a simple, secure, and user-friendly platform for both administrators and students.

The objectives of this project are:

- To replace the traditional paper-based notice board with a digital platform.
- To provide secure authentication for different user roles such as Super Admin, Admin, and User.
- To allow administrators to create, edit, update, and delete notices easily.
- To organize notices according to departments and categories for better management.
- To enable students to view notices anytime and from anywhere through the mobile application.
- To allow users to download PDF attachments and view uploaded notice images.
- To provide a dashboard showing important statistics such as total notices, users, departments, and categories.
- To reduce manual work and improve communication between the university administration and students.
- To develop a scalable application that can be extended with additional features in the future.

SCOPE OF THE PROJECT

The University Notice Management System is developed to simplify the process of managing and distributing university notices through a mobile application. The system is designed for educational institutions where notices are shared regularly with students and faculty members.

The project provides separate access for Super Admin, Admin, and Users. Super Admin has complete control over the system, including managing administrators, departments, categories, and notices. Admin users can manage notices and perform day-to-day operations. Students can securely log in, view notices, search for important announcements, and download attached PDF files.

The application supports image uploads, PDF attachments, role-based authentication, department-wise notice filtering, category management, dashboard statistics, dark mode, and profile management. These features make the system suitable for managing academic, examination, placement, hostel, and event notices.

Although the current system is designed as a mobile application, it can be expanded in the future by adding web support, push notifications, email alerts, cloud storage, and advanced analytics. The project provides a strong foundation for a complete digital notice management solution.

SOFTWARE AND HARDWARE REQUIREMENTS

5.1 Software Requirements

The University Notice Management System was developed using modern technologies to ensure good performance, maintainability, and scalability. The following software tools were used during the development of this project.

Software	Purpose
Flutter SDK	Mobile application development
Dart	Programming language for Flutter
Laravel	Backend framework for API development
PHP	Server-side programming language
MySQL	Database management system
XAMPP	Local development server (Apache & MySQL)
Visual Studio Code	Source code editor
Android Studio	Android SDK and Emulator
Postman	API testing
Git	Version control

Flutter

Flutter was used to develop the frontend of the application. It provides a single codebase for building modern, responsive, and high-performance mobile applications.

Laravel

Laravel was used to build the backend of the project. It provides REST APIs, routing, authentication, database migrations, and secure communication between the application and the database.

MySQL

MySQL was used as the relational database to store all project data including users, notices, departments, categories, and administrator information.

Visual Studio Code

Visual Studio Code was used as the primary code editor for developing both the Flutter frontend and Laravel backend.

XAMPP

XAMPP was used to run the Apache web server and MySQL database during local development.

Postman

Postman was used to test all REST APIs before integrating them with the Flutter application.

5.2 Hardware Requirements

The project was developed and tested on the following hardware configuration.

Hardware	Specification
Processor	Intel Core i5 (or equivalent)
RAM	8 GB
Storage	256 GB SSD
Operating System	Windows 11
Mobile Device	Android Smartphone
Internet Connection	Required for API communication

The application runs smoothly on Android devices and can be extended to support iOS with minor configuration changes because it is developed using Flutter.

TECHNOLOGY STACK

The University Notice Management System follows a client-server architecture where the Flutter application acts as the client, Laravel acts as the backend server, and MySQL stores all application data.

Frontend

The frontend was developed using Flutter and Dart. Flutter provides a rich collection of widgets that helped in creating responsive and modern user interfaces. Features such as login, registration, dashboard, notice listing, notice details, profile management, dark mode, and settings were implemented in Flutter.

Backend

The backend was developed using Laravel, a PHP framework. Laravel was responsible for handling API requests, authentication, CRUD operations, validation, and communication with the MySQL database.

Database

MySQL was used to store all application data. Tables such as Users, Notices, Departments, Categories, and Admins were created using Laravel migrations. All user information and notices are securely stored in the database.

REST APIs

REST APIs were developed using Laravel to connect the Flutter application with the backend. These APIs perform operations such as login, registration, creating notices, fetching notices, updating notices, deleting notices, and retrieving dashboard statistics.

API Testing

All APIs were tested using Postman before integrating them into the Flutter application. This ensured that every endpoint returned the expected response and handled errors correctly.

SYSTEM ARCHITECTURE

The University Notice Management System follows a client-server architecture. The application is divided into three main components: the Flutter mobile application, the Laravel backend, and the MySQL database.

The Flutter application acts as the client interface where users interact with the system. Students, Admins, and Super Admins can log in using their credentials and perform operations based on their assigned roles.

Whenever a user performs an action such as logging in, viewing notices, creating a notice, updating a profile, or downloading a PDF, the Flutter application sends an HTTP request to the Laravel backend through REST APIs.

The Laravel backend processes each request by validating the data, applying business logic, and communicating with the MySQL database. After processing the request, Laravel sends a JSON response back to the Flutter application.

The MySQL database stores all application data including users, notices, departments, categories, administrator information, images, and PDF file paths. Laravel uses Eloquent ORM to interact with the database efficiently.

This architecture separates the frontend, backend, and database into independent layers, making the application easier to maintain, update, and expand in the future.

System Flow

User → Flutter Application → REST API → Laravel Backend → MySQL Database

↓

JSON Response

↓

Flutter Application → Display Result to User

DATABASE DESIGN

The University Notice Management System uses MySQL as the relational database to store and manage application data. The database consists of multiple tables that are connected through relationships to ensure efficient data management.

Users Table

The Users table stores information about all registered users in the application. It includes authentication details and role information.

Main Fields:

- ID
- Name
- Email
- Password
- Role (Super Admin, Admin, User)
- Created At
- Updated At

Notices Table

The Notices table stores all notices published in the system.

Main Fields:

- ID
- Title
- Description
- Image
- PDF
- Department ID
- Category ID
- Priority
- Publish Date
- Expiry Date
- Created By

Each notice is linked to a department and category using foreign keys.

Departments Table

The Departments table stores all university departments.

Main Fields:

- ID
- Department Name
- Description

Examples:

- Computer Science
- Electronics
- Mechanical
- Civil

Categories Table

The Categories table stores notice categories.

Main Fields:

- ID
- Category Name
- Description

Examples:

- Academic
- Examination
- Placement
- Hostel
- Events

Relationships

The database uses relationships to maintain data consistency.

- One Department can have many Notices.
- One Category can have many Notices.
- One User (Admin/Super Admin) can create multiple Notices.
- Every Notice belongs to one Department and one Category.

Laravel Eloquent ORM manages these relationships using Model classes, making database operations simple and efficient.

BACKEND IMPLEMENTATION

The backend of the University Notice Management System was developed using the Laravel framework. Laravel was selected because it provides a clean project structure, built-in authentication support, database migrations, routing, validation, and REST API development. The backend is responsible for processing requests from the Flutter application, communicating with the MySQL database, and returning JSON responses.

Project Structure

The backend project follows Laravel's MVC (Model-View-Controller) architecture.

- **Models** are used to represent database tables.
- **Controllers** handle the application logic and API requests.
- **Routes** define the API endpoints.
- **Migrations** create and manage the database structure.
- **Storage** stores uploaded images and PDF files.

This architecture keeps the project organized and makes it easier to maintain.

Database Migrations

Laravel migrations were used to create the database tables instead of creating them manually. Each migration defines the structure of a table, including columns, data types, primary keys, foreign keys, and timestamps.

After creating the migration files, the following command was executed:

```
php artisan migrate
```

This command automatically created all the required tables in the MySQL database.

The main tables created for this project are:

- Users

- Notices
- Departments
- Categories

Using migrations makes the project portable because the database structure can be recreated on any system.

Models

Laravel Eloquent Models were used to interact with the MySQL database.

The main models used in the project are:

- User Model
- Notice Model
- Department Model
- Category Model

Each model represents a database table and performs CRUD operations without writing complex SQL queries.

Relationships were also defined between the models.

For example:

- One Department has many Notices.
- One Category has many Notices.
- One User can create multiple Notices.

Controllers

Controllers contain the business logic of the application.

The following controllers were developed:

AuthController

Responsible for:

- User Registration
- User Login
- User Logout

NoticeController

Responsible for:

- Create Notice
- View Notices
- Update Notice
- Delete Notice
- Upload Images
- Upload PDF Files

CategoryController

Responsible for:

- Add Category
- Update Category
- Delete Category
- Display Categories

DepartmentController

Responsible for:

- Add Department
- Edit Department
- Delete Department

- Display Departments

DashboardController

Responsible for displaying dashboard statistics such as:

- Total Users
- Total Notices
- Total Departments
- Total Categories
- Recent Notices

AdminController

Responsible for:

- Create Admin
- Manage Admins
- Promote User to Admin
- Remove Admin
- Update Admin Details

ForgotPasswordController

Responsible for:

- Sending OTP
- Verifying OTP
- Resetting Password

REST APIs

REST APIs were developed so that the Flutter application could communicate with the Laravel backend.

Some of the APIs developed include:

- Register API
- Login API
- Logout API
- Notice APIs
- Department APIs
- Category APIs
- Dashboard API
- User API
- Admin API
- Forgot Password API
- OTP Verification API
- Reset Password API

Each API receives a request from Flutter, processes it, interacts with the MySQL database, and returns a JSON response.

File Upload

The backend supports uploading both images and PDF files.

When an administrator creates a notice:

- Images are stored inside the Laravel Storage folder.
- PDF files are also stored securely.
- The file paths are saved in the MySQL database.

The Flutter application then retrieves these file paths through the APIs and displays the uploaded images or downloads the PDF files.

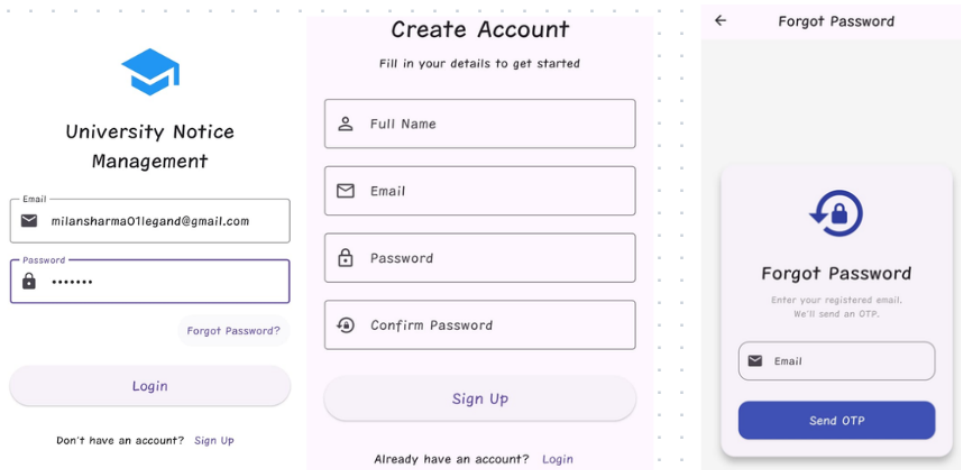
API Testing

Before integrating the backend with Flutter, every API was tested individually.

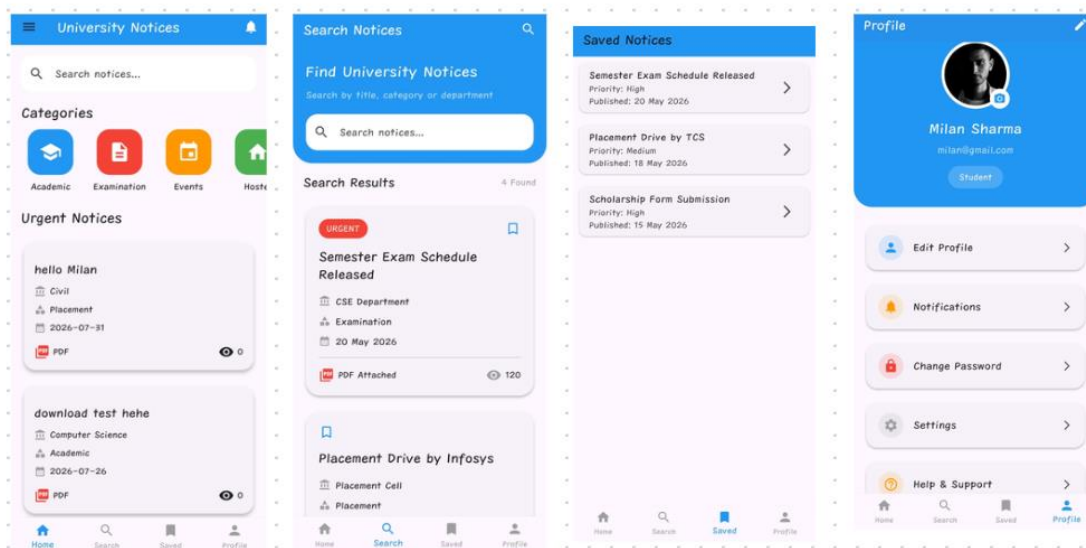
API testing ensured:

- Correct request handling
- Proper JSON responses
- Successful database operations
- Error handling for invalid requests

Testing each endpoint before integration reduced debugging time and improved the overall reliability of the application.



login page sing up page otp section

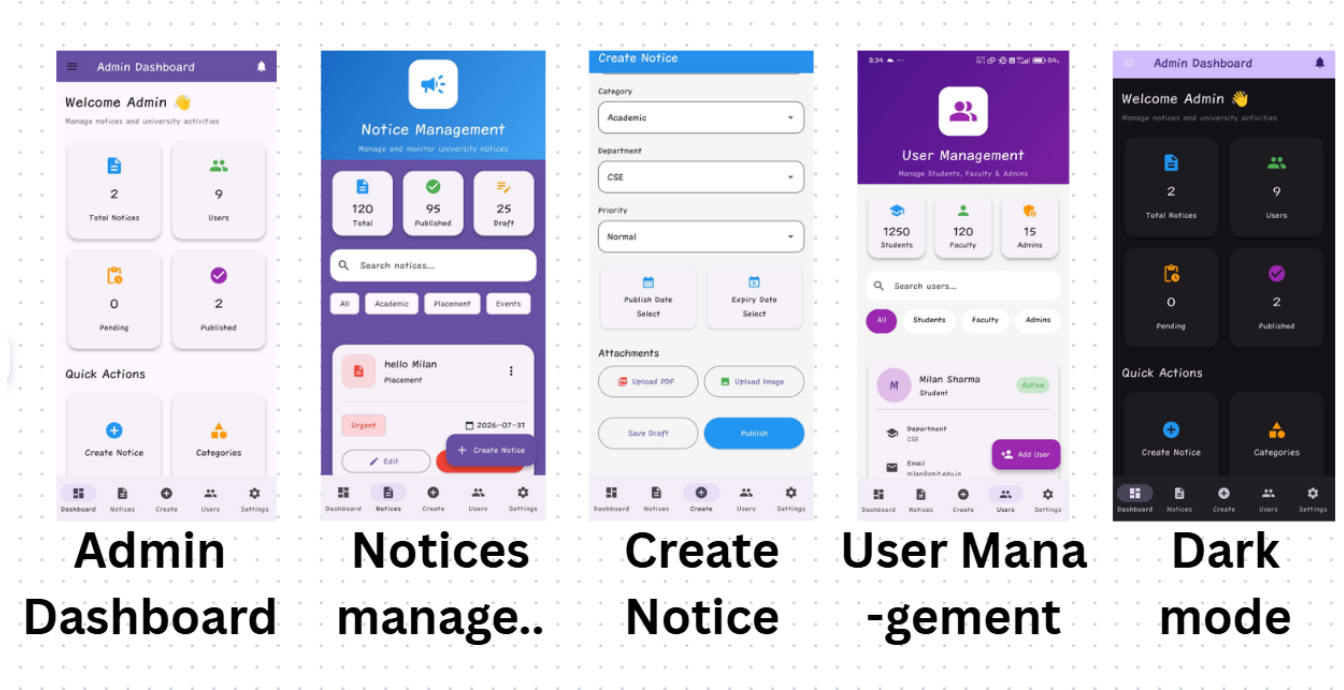


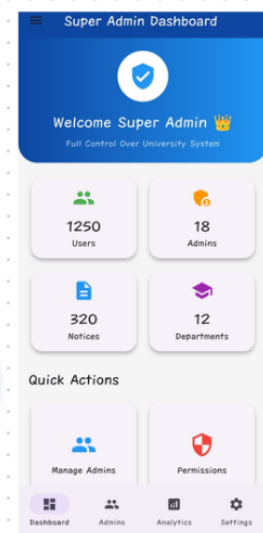
user
home page

Search page

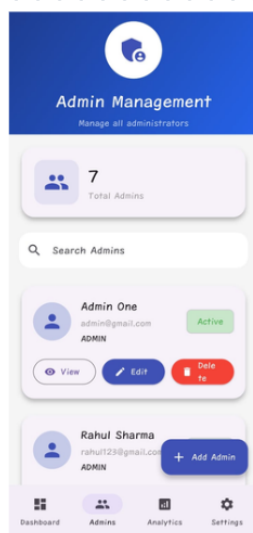
Save page

User profile
page





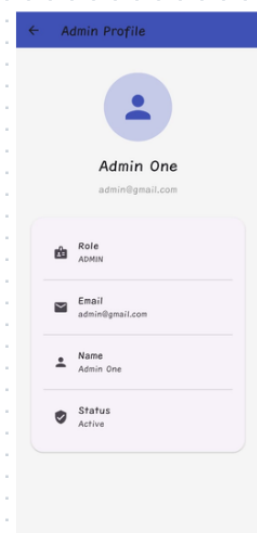
**SuperAdmin
Dashboard**



**Admin
manage..**



**Add
Admin**



**Admin
Profile**